Angular Advanced
Day 2 – short recap

Peter Kassenaar –
info@kassenaar.com

Yesterday:

- Subscribing to routing events
- HttpClientModule,
- RxJS 5.5 lettable operators
- State Management w/ @ngrx/store
  - Store
  - Actions
  - Reducers
  - Payload
- Testing

RxJS 5.5+: lettable operators en `.pipe()`

This repository Search Pull requests Issues Marketplace Explore

ReactiveX / rxjs

Watch 357 Star 9,548 Fork 905

Code Issues 225 Pull requests 68 Projects 3 Insights

Branch: master rxjs / doc / lettable-operators.md Find file Copy path

seniorquico docs: change "pipe" utility function module path from "utils" to "uti... 39c2e7c on Oct 25

6 contributors

237 lines (187 sloc) 9.12 KB Raw Blame History

Lettable Operators

Starting in version 5.5 we have shipped "lettable operators", which can be accessed in `rxjs/operators` (notice the pluralized "operators"). These are meant to be a better approach for pulling in just the operators you need than the "patch" operators found in `rxjs/add/operator/*`.

NOTE: Using `rxjs/operators` without making changes to your build process can result in larger bundles. See Known Issues section below.

Renamed Operators

Due to having operators available independent of an Observable, operator names cannot conflict with JavaScript keyword restrictions. Therefore the names of the lettable version of some operators have changed. These operators are:

https://github.com/ReactiveX/rxjs/blob/master/doc/lettable-operators.md

Testing repo:

https://github.com/PeterKassenaar/ng-testing

This repository Search Pull requests Issues Marketplace Explore

PeterKassenaar / ng-testing

Unwatch 1 Star 1 Fork 0

Code Issues 0 Pull requests 0 Projects 0 Wiki Insights Settings

Sample Angular-cli project with Jasmine/Karma unit tests

Edit

Add topics

12 commits 1 branch 0 releases 1 contributor

Branch: master New pull request Create new file Upload files Find file Clone or download

PeterKassenaar Update README.md Latest commit 704213d on Oct 27

e2e	small version bumps	a month ago
src	small version bumps	a month ago
.angular-cli.json	First commit of code, with sample tests.	9 months ago
.editorconfig	First commit of code, with sample tests.	9 months ago
.gitignore	Added Firefox launcher	5 months ago
README.md	Update README.md	a month ago
karma.conf.js	small version bumps	a month ago
package.json	small version bumps	a month ago
protractor.conf.js	First commit of code, with sample tests.	9 months ago
tsconfig.json	some small changes	4 months ago

Today

- Wrap-up testing
  - Testing & Mocking Http Services
- Smart component/View components
- More on Observable Operators
- Realtime databases with Firebase
- …

On RxJS operators

Multiple RxJS-operators in Action

.map() | .filter() | .scan() | .concat() | .merge() | .mergeMap() | .forkJoin() | Movie App | Movie App 2 |

.map() result - every item in array multiplied by 10.

See `/app/map/map.component.ts` for example code.

```
[
  10,
  20,
  30,
  40,
  50,
  60,
  70,
  80,
  90,
  100
]
```

"Apply a function/projection to each value of the source. Then emit the result."

More information: https://www.learnrxjs.io/operators/transformation/map.html

.mapTo() result

See `/app/map/map.component.ts` for example code.

```
[
  "Hello World",
  "Hello World",
  "Hello World",
  "Hello World",
  "Hello World",
  "Hello World",
  "Hello World",
  "Hello World",
  "Hello World",
  "Hello World"
]
```

" Map every emitted item to a constant value."

More information: https://www.learnrxjs.io/operators/transformation/mapto.html

https://github.com/PeterKassenaar/ng-rxjs-operators

Official documentation…

RxJS Home Manual Reference Source Test dark theme light theme

- AsyncSubject
- BehaviorSubject
- Notification
- Observable
- ReplaySubject
- Scheduler
- AnonymousSubject
- Subject
- SubjectSubscriber
- Subscriber
- Subscription
- ObservableInput
- Observer
- SubscribableOrPromise
- TeardownLogic
- Rx.Scheduler
- Rx.Symbol

observable
- ConnectableObservable

observable/dom
- AjaxError
- AjaxResponse
- AjaxTimeoutError
- AjaxRequest

operator
- GroupedObservable

scheduler
- Action

```
import {Observable} from '@reactivex/rxjs/es6/Observable.js'
```

public class | source

Observable

Direct Subclass:
ConnectableObservable, GroupedObservable, Subject

Indirect Subclass:
AnonymousSubject, AsyncSubject, BehaviorSubject, es6/operator/windowTime.js~CountedSubject, ReplaySubject

A representation of any set of values over any amount of time. This the most basic building block of RxJS.

Test:
Observable
Observable.create
Observable.lift

Static Method Summary

Static Public Methods	
public static	bindCallback(func: function, selector: function, scheduler: Scheduler): function(...params: *): Observable Converts a callback API to a function that returns an Observable.
public static	bindNodeCallback(func: function, selector: function, scheduler: Scheduler): function(...params: *): Observable

Generated by ESDoc(0.4.8)

http://reactivex.io/rxjs/class/es6/Observable.js~Observable.html

But (far) more readable

Type to search

learn-rxjs

LEARN RXJS

Introduction

Operators

Combination

combineAll

combineLatest

concat

concatAll

forkJoin

merge

mergeAll

pairwise

race

startWith

withLatestFrom

zip

Conditional

EDIT THIS PAGE

Star 733

Watch 54

Learn RxJS

Clear examples, explanations, and resources for RxJS.

Introduction

RxJS is one of the hottest libraries in web development today. Offering a powerful, functional approach for dealing with events and with integration points into a growing number of frameworks, libraries, and utilities, the case for learning Rx has never been more appealing. Couple this with the ability to utilize your knowledge across nearly any language, having a solid grasp on reactive programming and what it can offer seems like a no-brainer.

But...

Learning RxJS and reactive programming is hard. There's the multitude of concepts, large API surface, and fundamental shift in mindset from an imperative to declarative style. This site focuses on making these concepts approachable, the examples clear and easy to explore, and features references throughout to the best RxJS related material on the web. The goal is to supplement the official docs and pre-existing learning material while offering a new, fresh perspective to clear any hurdles and tackle the pain points. Learning Rx may be difficult but it is certainly worth the effort!

Content

https://www.learnrxjs.io/

Article - 6 Operators you must know

Sign in / Sign up

Netanel Basal Follow

Jan 24 · 3 min read

RxJS—Six Operators That you Must Know

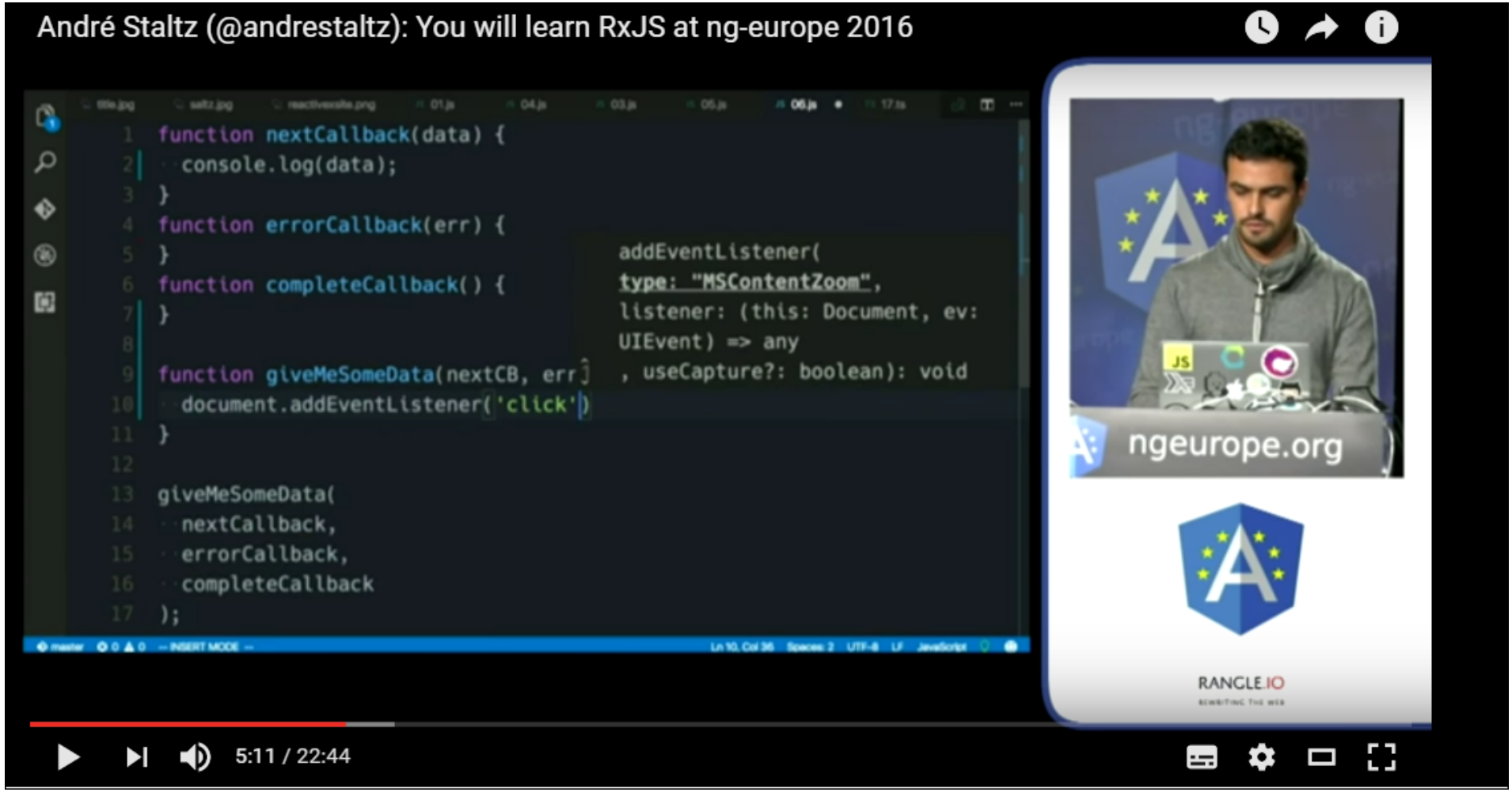
```
class TakeSubscriber<T> extends Subscriber<T> {
  private count: number = 0;

  constructor(destination: Subscriber<T>, private total: number) {
    super(destination);
  }

  protected _next(value: T): void {
    const total = this.total;
    const count = ++this.count;
    if (count <= total) {
```

Never miss a story from **NetanelBasal**, when you sign up for Medium. Learn more

GET UPDATES

https://netbasal.com/rxjs-six-operators-that-you-must-know-5ed3b6e238a0#.11of73aox

Creating Observables from scratch - André Staltz



https://www.youtube.com/watch?v=uQ1zhJHclvs

GitHub Gist Search... All gists GitHub New gist

staltz / introrx.md
Last active an hour ago

Star 10,812 Fork 1203

Code Revisions 259 Stars 10812 Forks 1203 Embed <script src="https://gist. Download ZIP

The introduction to Reactive Programming you've been missing

introrx.md Raw

The introduction to Reactive Programming you've been missing

(by @andrestaltz)

This tutorial as a series of videos

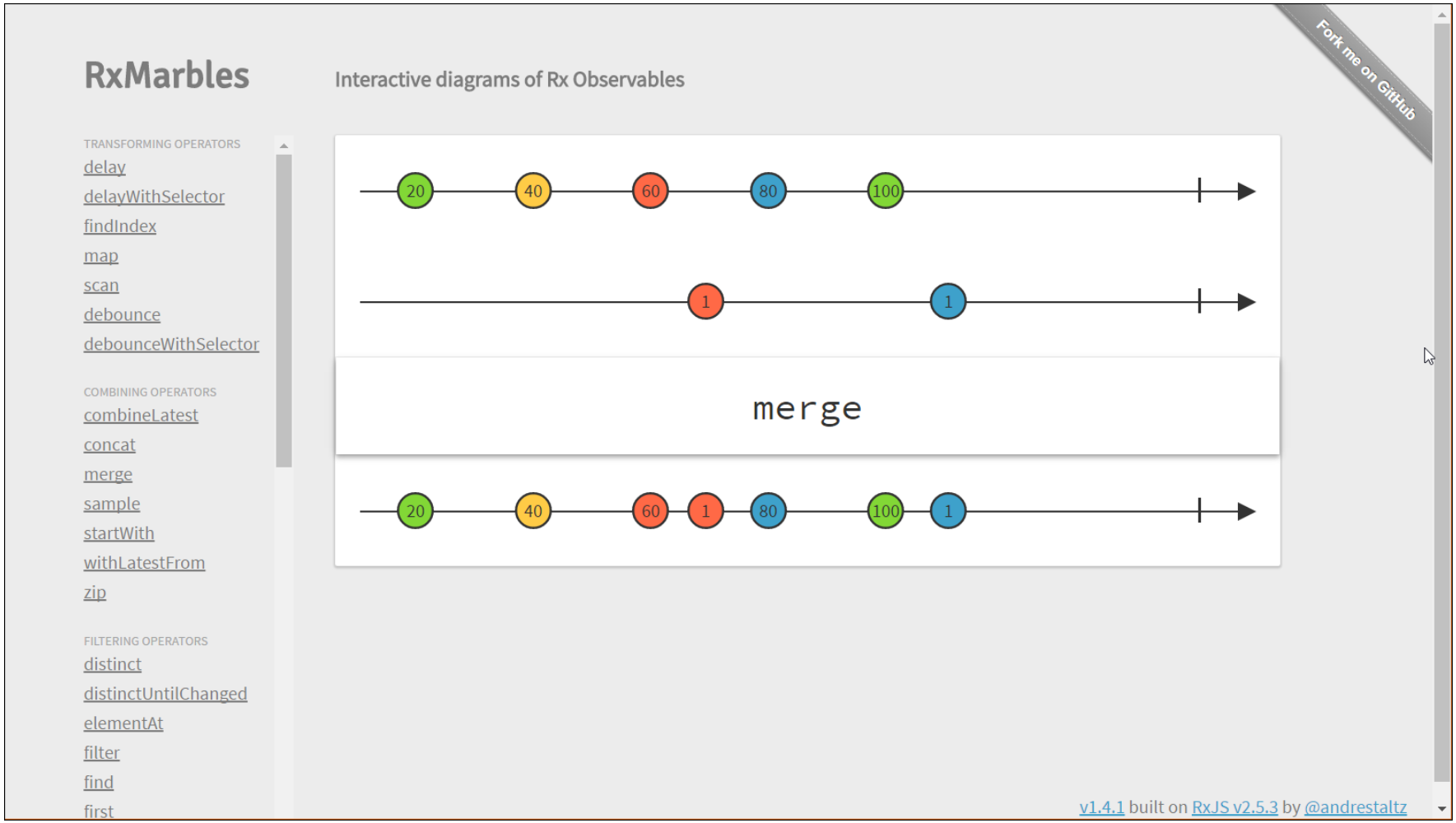
If you prefer to watch video tutorials with live-coding, then check out this series I recorded with the same contents as in this article: Egghead.io - Introduction to Reactive Programming.

So you're curious in learning this new thing called Reactive Programming, particularly its variant comprising of Rx, Bacon.js, RAC, and others.

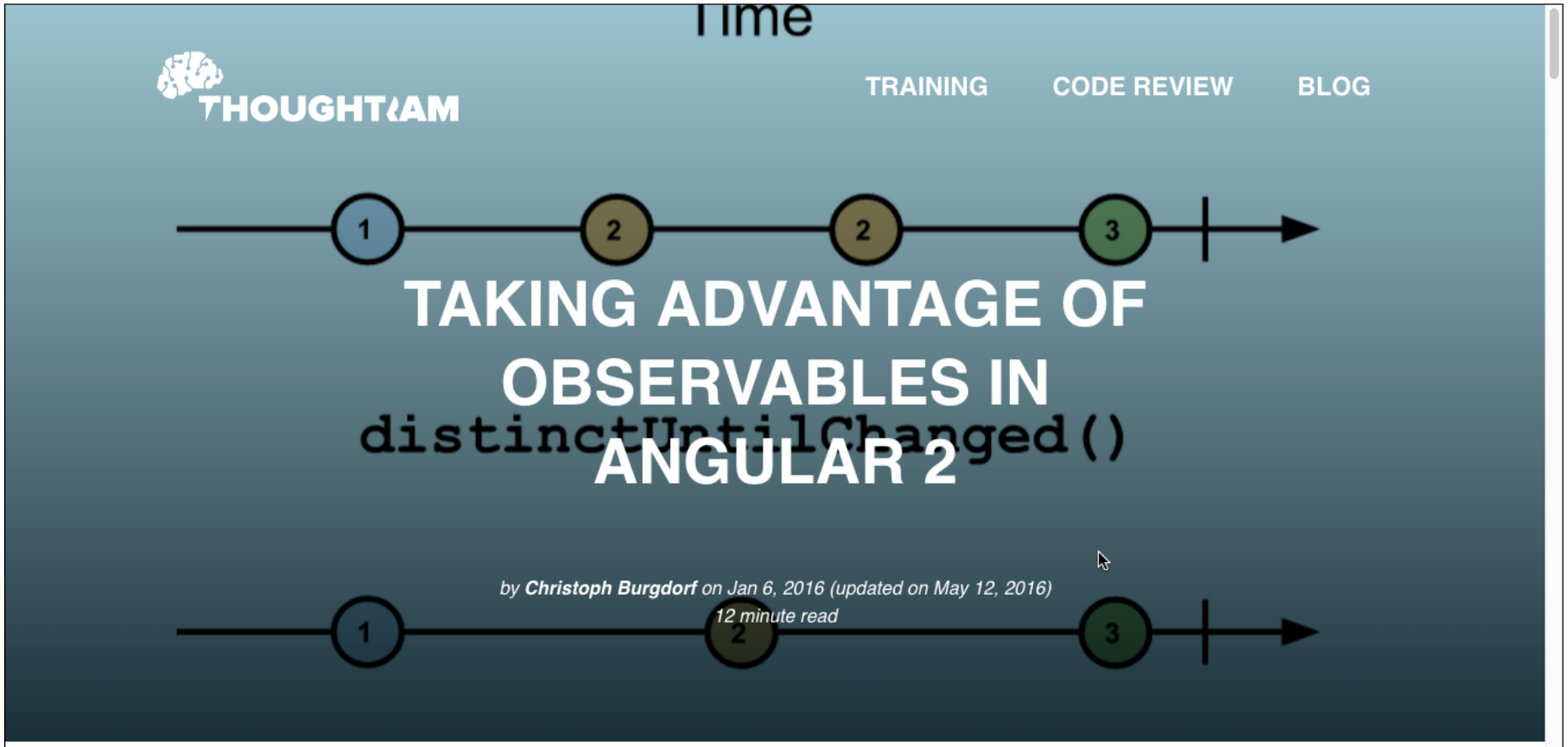
Learning it is hard, even harder by the lack of good material. When I started, I tried looking for tutorials. I found only a handful of practical guides, but they just scratched the surface and never tackled the challenge of building the whole architecture

https://gist.github.com/staltz/868e7e9bc2a7b8c1f754

Also by Andre Stalz - RxMarbles



http://rxmarbles.com/



Some people seem to be confused why Angular 2 seems to favor the Observable abstraction over the Promise abstraction when it comes to dealing with async behavior.

There are pretty good resources about the difference between Observables and Promises already out there. I especially like to highlight this free 7 minutes video by Ben Lesh on egghead.io. Technically there are a couple of obvious differences like the *disposability* and *lazyness* of Observables. In this article we like to focus on some practical advantages that

http://blog.thoughtram.io/angular/2016/01/06/taking-advantage-of-observables-in-angular2.html

https://www.youtube.com/watch?v=65Us8NwmYf4

Wrap up

- Angular CLI tips & tricks
- Content projection
- NG applications with multiple modules
  - Routing and Lazy loading modules
  - Loading strategies
- Data flow with `@Input()` and `@Output()`
- Managing state with `@ngrx/store`
- Testing
- Pattern: Smart components/View components
- Observable operators
- Angular and realtime databases/Firebase

www.angulartraining.nl